PHASE 2 OF 8

Architecture & Project Structure

How the browser, FastAPI backend, Gemini modules and local model fit together — and where every file lives.

Layers | Request Lifecycle | State | File Layout

2.1 High-Level Architecture

The browser talks **only** to the FastAPI backend — it never calls Google's servers directly. The backend then decides, per feature, whether to call Google's Gemini API over the internet or run a small language model locally on the same machine.

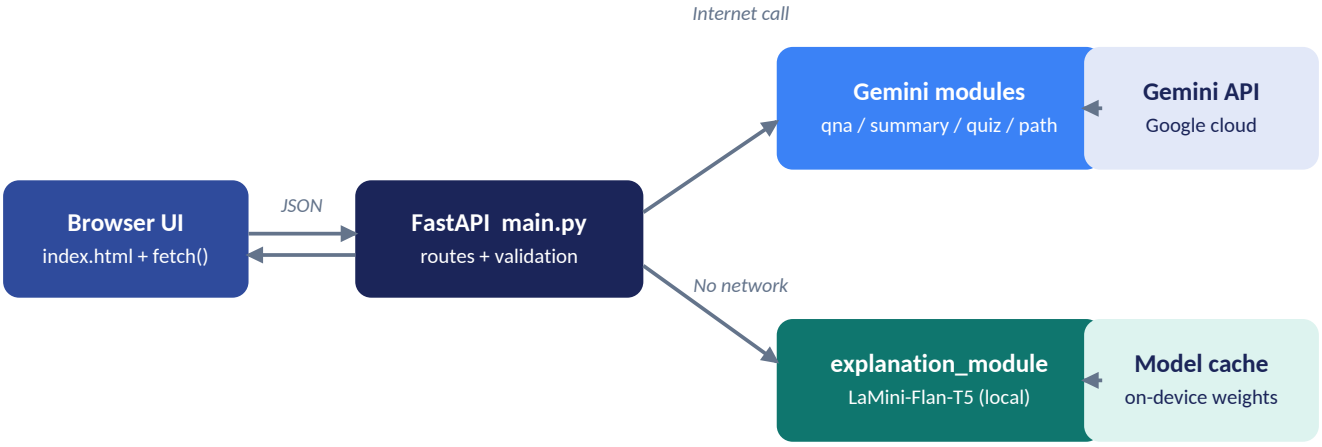


Figure — EduGenie system overview: one backend, two AI engines.

Layer	Responsibility
Browser (UI)	Renders `templates/index.html`; collects form input; calls backend endpoints via `fetch()`.
FastAPI app (main.py)	Defines every route; validates input; dispatches to the right module.
Gemini-backed modules	`qna.py`, `summary_module.py`, `quiz_module.py`, `learning_path.py` — each builds a prompt and calls the Gemini API.
Local model module	`explanation_module.py` — loads `MBZUAI/LaMini-Flan-T5-783M` once and runs inference on-device.

2.2 Request Lifecycle

Every feature follows the same six-step journey. The example below traces a student asking a question.

Step	What happens	Where
1	The user types a question into the Q&A field.	Browser
2	The page's JavaScript sends a GET/POST request (`fetch`) to the backend.	Browser → FastAPI
3	`main.py` routes the request to the matching module, which calls Gemini or the local model.	FastAPI
4	Gemini (or the local model) returns text or JSON.	AI engine
5	The backend wraps the result in a JSON response.	FastAPI
6	The frontend JavaScript renders the answer on the page.	Browser

2.3 Feature Routing Map

Feature	Route	Module	Engine
Home page	`GET /`	templates/index.html	—
Q&A	`GET /qa`	qna.py	Gemini
Explain	`POST /explain`	explanation_module.py	Local model
Summarize	`POST /summarize`	summary_module.py	Gemini
Quiz	`POST /quiz`	quiz_module.py	Gemini
Learning path	`GET /learn/recommendations`	learning_path.py	Gemini

2.4 Where State Lives

EduGenie, as documented, is **stateless between requests**: each call is handled independently, with no session or database layer. The only persistent state on disk is the cached local-model weights (downloaded once by `explanation_module.py`) and whatever the browser itself remembers about the current page.

Advantage of stateless design	Trade-off to be aware of
Simple to reason about and debug	No memory of a user's previous questions
Easy to scale horizontally later	No quiz history or progress tracking
No database to install or secure	Repeated identical requests are recomputed

2.5 Project Structure

The project is a flat FastAPI application with one module per feature.

```
EduGenie/
+-- main.py                # FastAPI app & all routes
+-- explanation_module.py   # Local model (LaMini-Flan-T5) for Explain
+-- qna.py                 # Gemini-powered Q&A
+-- quiz_module.py         # Gemini quiz generation + JSON parsing
+-- summary_module.py      # Gemini-powered summarization
+-- learning_path.py       # Gemini learning path / roadmap
+-- check_models.py        # Utility: lists Gemini models for your key
+-- templates/
|   +-- index.html         # Frontend page (forms + JS)
+-- static/
|   +-- style.css          # Styling for the frontend
+-- requirements.txt       # Python dependencies
+-- .env                   # GEMINI_API_KEY (never committed)
+-- README.md              # Setup & usage notes
```

Double-check before running

`templates/` and `static/` are referenced by `main.py` (it mounts `/static` and renders `index.html`). Confirm both folders exist in the project before starting the server.

2.6 File Responsibilities

File	Owns	Depends on
main.py	Routes, input validation, template rendering, static mount	All feature modules
qna.py	Free-form question answering	google-genai, python-dotenv
summary_module.py	Plain-language summaries	google-genai, python-dotenv
quiz_module.py	Quiz prompt, JSON cleaning and validation	google-genai, re, json
learning_path.py	Leveled roadmap prompt and response handling	google-genai, python-dotenv
explanation_module.py	Local model loading, caching, generation	transformers, torch
check_models.py	Lists models usable with your key	google-genai

2.7 Extension Pattern

Adding a capability (for example, flashcards) follows a repeatable recipe: create a new module file with one function that builds a prompt and calls Gemini or a local model, then add a single route in `main.py` that calls it — mirroring how `quiz_module.py` and `generate_quiz()` are wired today.